University of Information Technology

2025–2026 Academic Year

CST-4104 (Artificial Intelligence)

AstroLogic

A Prolog-Driven Symbolic Reasoning System for Zodiac Profiling, Tarot Interpretation, and Compatibility Analysis

Submitted by Group – 6

Section (A), Semester IV

Department of Computer Science and Technology

September, 2026

Group Members

Student ID	Name	Email
TNT-2384	La Yaung Htut	layaunghtut@uit.edu.mm
TNT-2331	Bhone Myat Khaing	bhonemyatkhaing@uit.edu.mm
TNT-2347	Aye Thandar Kyaw	atdkyaw@uit.edu.mm
TNT-2354	May Myat Noe Latt	mmnlatt@uit.edu.mm
TNT-2338	Pwint Thazin Myint	ptzmyint@uit.edu.mm
TNT-2414	Khaing Yadanar	khaingyadanar@uit.edu.mm

Roles and Responsibilities

Name	Description
Bhone Myat Khaing	Prolog knowledge base design – zodiac profile rules, trait derivation, and reasoning trace generation (Module 1).
May Myat Noe Latt	Prolog card-selection module – tarot theme classification, eligibility filtering, and spread ranking (Module 2).
Pwint Thazin Myint	Prolog reading-analysis and synastry modules – theme/conflict detection and weighted compatibility scoring (Modules 3 & 4).
La Yaung Htut	FastAPI backend integration – PrologService bridge, OpenRouter LLM orchestration, REST API endpoints.
Khaing Yadanar	SvelteKit frontend – UI/UX design, page routing, reasoning-trace visualization.
Aye Thandar Kyaw	Testing, documentation, and database design – SQLite schema, API docs, report compilation.

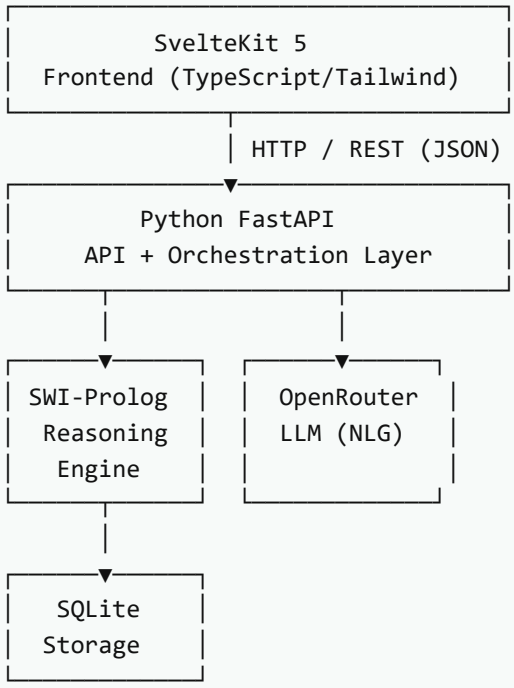
Contents

1. Introduction

AstroLogic is a full-stack, AI-assisted web application that reinterprets Western zodiac astrology and tarot practice as a **symbolic knowledge-representation and reasoning problem**. Rather than treating horoscopes and tarot readings as arbitrary text generated by a language model, AstroLogic encodes the domain – zodiac signs, elements, modalities, ruling planets, personality traits, the 78-card tarot deck, spreads, and compatibility rules – as a formal **Prolog knowledge base**. A large language model (LLM) is used only at the very last step, to translate the structured symbolic output produced by Prolog into natural, readable language. This design keeps the "reasoning" auditable, deterministic, and explainable, while still producing an engaging, conversational user experience.

The system is organized as a three-tier architecture:

- **SvelteKit 5 frontend** (TypeScript, Tailwind CSS) – the interactive web client, including a dashboard, tarot reading flow, horoscope generator, compatibility checker, an AI chat assistant, and a reasoning-trace visualizer.
- **Python FastAPI backend** – the orchestration and API layer that classifies user questions, queries the Prolog engine, calls the OpenRouter LLM service, and persists history to SQLite.
- **SWI-Prolog knowledge base** – the symbolic reasoning core: zodiac facts, tarot facts, and four dedicated reasoning modules that derive profiles, select cards, analyze readings, and score compatibility.



The core design principle is a strict **separation of concerns**: Prolog decides *what* the system reasons (facts, rules, and inference), Python decides *how* the system orchestrates that reasoning, the LLM decides *how* the result is expressed in natural language, and SvelteKit decides *how* the user experiences all of it. If the LLM provider is unreachable, the application degrades gracefully to deterministic, Prolog-derived interpretations rather than failing outright.

> AstroLogic is framed explicitly as a **reflective / entertainment** tool. The system prompt and UI copy avoid claims of scientific prediction or medical/psychological diagnosis; the Prolog rules model traditional astrological/tarot symbolism, not empirical claims about personality or the future.

2. Objectives

The objectives below define the scope of Version 1 (V1) of AstroLogic.

1. **Model zodiac knowledge symbolically.** Represent all 12 zodiac signs with their elements, modalities, ruling planets, symbolic traits, and date ranges as Prolog facts, and derive strengths, challenges, and full personality profiles through inference rather than hard-coded text.
2. **Model the tarot deck symbolically.** Represent the full 78-card deck (22 Major Arcana + 56 Minor Arcana) with keywords, upright/reversed meanings, and thematic categories, and use these facts to drive card selection.
3. **Provide context-aware card selection.** Classify a user's free-text question into a category (e.g. love, career, decision) and use Prolog rules to filter eligible cards and recommend an appropriate spread (single card, three-card, Celtic Cross, decision spread, etc.).
4. **Produce explainable readings.** Analyze a set of drawn cards for recurring themes, dominant elements/suits, and conflicting or complementary themes, and expose the underlying rule firings as a **reasoning trace** the user can inspect.
5. **Score zodiac compatibility transparently.** Compute a weighted compatibility score between two signs from four independent factors (element, modality, trait, and planetary symbolism) rather than a single opaque lookup table.
6. **Layer natural-language generation on top of symbolic output.** Use an LLM (via OpenRouter) purely as a presentation layer that narrates Prolog's structured facts, with a deterministic fallback when the LLM is unavailable.
7. **Deliver the reasoning through a usable web application.** Build a SvelteKit interface (dashboard, zodiac explorer, tarot reading, horoscope, compatibility, chat, history, analytics) backed by a documented REST API.

3. Requirements

3.1 Functional Requirements

ID	Requirement	Description
FR-01	Zodiac profile lookup	User selects/enters a birth date; system returns sign, element, modality, ruling planet, traits, strengths and challenges.
FR-02	Tarot card draw	System draws N cards for a chosen spread and returns card identity, orientation, and position-aware meaning.
FR-03	Question classification	Free-text question is classified into a category to drive spread recommendation and card eligibility.
FR-04	Reading analysis	System detects dominant themes, elements, arcana balance, and theme conflicts across a multi-card reading.
FR-05	Horoscope generation	System generates a themed daily horoscope (theme, guidance, reflection, opportunity, caution) from zodiac + mood input.
FR-06	Compatibility analysis	System scores compatibility between two zodiac signs with a full component breakdown and explanation.
FR-07	AI chat assistant	Conversational endpoint answers questions about signs, cards, and readings using Prolog context + LLM narration.
FR-08	Reasoning-trace visualization	Every Prolog inference exposes a structured trace (rule, input, result, explanation) renderable in the UI.
FR-09	History & persistence	Readings and chat sessions are stored in SQLite and retrievable/deletable via the History page.
FR-10	Graceful LLM fallback	If OpenRouter is unreachable or times out (30s), the system serves deterministic Prolog-derived text.

3.2 Non-Functional Requirements

- **Explainability** – every symbolic conclusion must be traceable to the specific Prolog rule(s) that produced it.
- **Separation of concerns** – the LLM must never perform core reasoning; it only narrates facts supplied by Prolog.
- **Resilience** – the system must degrade gracefully (fallback templates) rather than fail when the LLM provider is down.
- **Responsiveness** – REST endpoints should respond within a few seconds for typical queries; LLM calls are bounded by a 30s timeout.
- **Maintainability** – the Prolog knowledge base is split into cohesive modules (base facts vs. reasoning modules) to keep rules easy to extend.
- **Ethical framing** – content must present itself as reflective/entertainment, avoiding predictive or diagnostic claims.

3.3 Technology Stack

Layer	Technology
Frontend	SvelteKit 5, TypeScript, Tailwind CSS, Vite, Vitest
Backend / API	Python, FastAPI, Pydantic schemas
Symbolic Reasoning	SWI-Prolog knowledge base (13 modules, ~4,480 lines of rules/facts)
Natural-Language Generation	OpenRouter LLM API (e.g. GPT-3.5-class model)
Persistence	SQLite (schema.sql), Docker Compose for local orchestration
Tooling	ESLint, Prettier, pytest, svelte-check

4. Algorithms

4.1 End-to-End Reasoning Pipeline

Every user-facing request (tarot reading, horoscope, compatibility check, chat message) follows the same seven-step pipeline:

1. User input is captured by the SvelteKit frontend and sent to FastAPI over REST.
2. FastAPI performs lightweight NLP to classify the question (e.g. love / career / decision / general).
3. FastAPI's `PrologService` queries SWI-Prolog with the classified inputs.
4. Prolog executes rule-based inference and returns **structured symbolic results** (facts, scores, traces) – never free text.
5. FastAPI forwards the structured results to OpenRouter with a constrained system prompt.
6. OpenRouter returns natural-language narration grounded in those facts (or, on failure, FastAPI substitutes a deterministic template built directly from the Prolog output).
7. The combined response (symbolic data + narrative + reasoning trace) is returned to SvelteKit and rendered to the user.

4.2 Zodiac Profile Derivation

Given a sign, `generate_profile/2` composes four independent derivations – base facts (element, modality, ruling planet, traits), an element-personality mapping, a modality-approach mapping, and a planetary-influence mapping – into one profile term. Strengths and challenges are derived per-trait via `trait_strength/2` and `trait_challenge/2`, so a sign's five traits expand into five strengths and five challenges automatically, without any sign-specific hard-coding. Birth-date → sign mapping (`zodiac_from_date/3`) uses ordered date-boundary rules with cut-fail so exactly one sign is returned per date.

4.3 Question Classification and Card Selection

A free-text question is classified into a category (love, career, decision, general, …). The category, together with the user's zodiac sign, forms a *context* that is passed through a filtering pipeline: `card_matches_theme/2`, `card_matches_question/2`, `card_matches_element/2`, and `card_matches_sign/2` each contribute a boolean signal; `card_eligible/2` combines them, `select_eligible_cards/3` produces the eligible set, and `card_priority/3` + `rank_cards/3` order candidates before the top N are drawn for the recommended spread.

4.4 Reading Analysis (Multi-Card Interpretation)

`analyze_reading/3` aggregates per-card analyses across the whole spread: it collects each card's themes via `reading_themes/2`, determines the `dominant_theme/2` by frequency, checks every theme pair against a `theme_conflict/3` table (e.g. *action* vs. *patience*, *independence* vs. *cooperation*) to surface tension in the reading, and separately flags complementary themes. Each card position (past, present, future, challenge, guidance, …) applies its own interpretive lens through `position_meaning_modifier/2`, so the same card yields a different interpretation depending on where it lands in the spread.

4.5 Weighted Compatibility Scoring

`zodiac_compatibility/3` is a genuine multi-factor weighted algorithm rather than a static lookup table:

Factor	Weight	Basis
Element compatibility	35%	Fire/Air vs. Water/Earth affinity table
Modality compatibility	20%	Cardinal / Fixed / Mutable interaction
Trait compatibility	30%	Count of complementary trait pairs between the two signs
Planetary symbolism	15%	Ruling-planet relationship

The four component scores are combined as `Overall = 0.35·E + 0.20·M + 0.30·T + 0.15·P`, then thresholded into `high` (≥75), `moderate` (≥50), or `low` levels. Because every component is independently queryable, the system can also produce a full **score breakdown** and a natural-language **explanation** for transparency.

4.6 Fallback Algorithm

Before any OpenRouter call, FastAPI starts a 30-second timeout. On timeout, network error, or malformed response, the orchestration layer substitutes a template-based generator that consumes the same structured Prolog output the LLM would have received (themes, scores, traits) and fills a deterministic sentence template – guaranteeing the user always receives a complete, non-empty response.

5. Prolog Codes

The knowledge base contains 13 files (~4,480 lines total) organized as two base fact libraries and four functional reasoning modules, orchestrated by a single entry point.

File	Role	Purpose
`zodiac.pl`	Base facts	12 signs, elements, modalities, ruling planets, traits, date ranges, symbols (110 lines)
`tarot.pl`	Base facts	78-card deck: arcana, keywords, upright/reversed meanings, themes (470 lines)
`zodiac_profile.pl`	Module 1	Profile generation, strengths/challenges, birth-date mapping, element/modality compatibility (709 lines)
`card_selection.pl`	Module 2	Card theming, eligibility filtering, spread selection and ranking (817 lines)
`reading_analysis.pl`	Module 3	Per-card and multi-card analysis, theme/conflict detection, advice (743 lines)
`synastry.pl`	Module 4	Weighted compatibility scoring and synastry explanation (495 lines)
`horoscope_rules.pl`	Support	Mood → theme mapping for horoscope generation (164 lines)
`spread_rules.pl`	Support	Question classification → spread recommendation (179 lines)
`recommendation_rules.pl`	Support	Card recommendation heuristics (195 lines)
`reasoning.pl`	Support	Cross-module orchestration (147 lines)
`compatibility.pl`	Support	Legacy/simple compatibility helpers (152 lines)
`history_analysis.pl`	Support	Aggregate analysis over stored reading history (215 lines)

File	Role	Purpose
`main.pl`	Entry point	Loads all modules; exposes convenience queries (84 lines)

5.1 Base Facts – Zodiac Knowledge

```
% --- Elements ---
element(aries, fire).
element(taurus, earth).
element(gemini, air).
element(cancer, water).
...

% --- Modalities ---
modality(aries, cardinal).
modality(taurus, fixed).
modality(gemini, mutable).
...

% --- Symbolic Traits ---
zodiac_traits(aries, [courage, initiative, independence,
                      impulsiveness, leadership]).
zodiac_traits(taurus, [patience, reliability, sensuality,
                       stubbornness, determination]).
...

% --- Helper: Get all signs of a given element ---
signs_of_element(Element, Signs) :-
    findall(Sign, element(Sign, Element), Signs).
```

Excerpt from `prolog/zodiac.pl` – declarative facts for all 12 signs.

5.2 Module 1 – Zodiac Profile Generation

```
:- use_module(zodiac).

% --- get_element/2 ---
get_element(Sign, Element) :-
    zodiac(Sign),
    element(Sign, Element).

% --- Trait strength / challenge mapping ---
trait_strength(initiative, 'Takes action and leads').
trait_challenge(initiative, 'May act impulsively without thinking').
...

% --- generate_profile/2 ---
% Combines base facts + element personality + modality
% approach + planetary influence into one profile term.
generate_profile(Sign, Profile) :-
    zodiac_profile(Sign, Base),
    element_personality(Sign, ElementDesc),
    modality_approach(Sign, ModalityDesc),
    planetary_influence(Sign, PlanetDesc),
    Profile = Base.put(_{
        element_personality: ElementDesc,
        modality_approach: ModalityDesc,
        planetary_influence: PlanetDesc
    }).
```

Excerpt from `prolog/zodiac_profile.pl` – every trait maps to a strength and a challenge, so profiles are derived, not hand-written per sign.

5.3 Module 2 – Card Theme Classification & Selection

```
:- use_module(zodiac).
:- use_module(tarot).
:- use_module(zodiac_profile).

% --- Card Theme Classification ---
card_theme(the_fool, new_beginnings).
card_theme(the_fool, spontaneity).
card_theme(the_hermit, reflection).
card_theme(the_hermit, solitude).
card_theme(the_hermit, introspection).
...

% --- Eligibility pipeline ---
card_matches_theme(Card, Theme)    :- card_theme(Card, Theme).
card_matches_question(Card, Cat)   :- ... .
card_eligible(Card, Context) :-
    card_matches_question(Card, Context.category),
    ( card_matches_sign(Card, Context.sign) ; true ).

select_eligible_cards(Context, AllEligible, Filtered) :-
    findall(C, card_eligible(C, Context), AllEligible),
    rank_cards(AllEligible, Context, Filtered).
```

Excerpt from `prolog/card_selection.pl` *– each of the 78 cards is tagged with multiple themes, then filtered/ranked per question context.*

5.4 Module 4 – Weighted Synastry Scoring

```
:- use_module(zodiac).
:- use_module(zodiac_profile).
:- use_module(card_selection).

% zodiac_compatibility/3
% Derived from element + modality + traits + planet,
% NOT a single hardcoded table.
zodiac_compatibility(Sign1, Sign2, Result) :-
    Sign1 \= Sign2,
    element(Sign1, E1), element(Sign2, E2),
    modality(Sign1, M1), modality(Sign2, M2),
    ruling_planet(Sign1, P1), ruling_planet(Sign2, P2),
    element_compatibility_score(E1, E2, EScore),
    modality_compatibility_score(M1, M2, MScore),
    trait_compatibility_score(Sign1, Sign2, TScore),
    planetary_compatibility_score(P1, P2, PScore),
    OverallScore is (EScore * 0.35) + (MScore * 0.20)
                  + (TScore * 0.30) + (PScore * 0.15),
    (   OverallScore >= 75 -> Level = high
    ;   OverallScore >= 50 -> Level = moderate
    ;   Level = low
    ),
    Result = compatibility_result{
        sign1: Sign1, sign2: Sign2, level: Level,
        score: OverallScore, element_score: EScore,
        modality_score: MScore, trait_score: TScore,
        planetary_score: PScore
    }.

element_compatibility_score(fire, fire, 90) :- !.
element_compatibility_score(fire, air, 85)  :- !.
element_compatibility_score(air, fire, 85)  :- !.
element_compatibility_score(air, air, 80)   :- !.
element_compatibility_score(earth, earth, 85) :- !.
```

Excerpt from `prolog/synastry.pl` *– a genuinely weighted, multi-factor scoring rule with cut-guarded lookup clauses.*

5.5 Entry Point and Module Loading

```
:- use_module(zodiac).
:- use_module(tarot).
:- use_module(zodiac_profile).
:- use_module(card_selection).
:- use_module(reading_analysis).
:- use_module(compatibility).
:- use_module(synastry).
:- use_module(horoscope_rules).
:- use_module(spread_rules).
:- use_module(recommendation_rules).
:- use_module(reasoning).
:- use_module(history_analysis).

% Get zodiac info
zodiac_info(Sign, Info) :-
    zodiac(Sign),
    element(Sign, Element),
    modality(Sign, Modality),
    ruling_planet(Sign, Planet),
    zodiac_traits(Sign, Traits),
    zodiac_date_range(Sign, DateRange),
    zodiac_symbol(Sign, Symbol),
    Info = zodiac_info{
        sign: Sign, element: Element, modality: Modality,
        ruling_planet: Planet, traits: Traits,
        date_range: DateRange, symbol: Symbol
    }.
```

Excerpt from `prolog/main.pl` *– single load point for all 12 modules plus convenience queries used directly by* `PrologService`.

5.6 Reasoning Trace Format

Every module returns machine-readable trace entries the frontend renders as a step-by-step explanation of the inference:

```
[
  {
    "rule": "element(aries, fire)",
    "input": "aries",
    "result": "Aries belongs to fire element",
    "explanation": "Fire element defines personality style"
  }
]
```

6. Current Limitations

- **Sun-sign only astrology.** V1 computes only the Sun sign from a birth date. Moon sign and Rising (Ascendant) sign require birth time and location and are *not* calculated in the current version, even though `chart_profile/4` already reserves fields for them.
- **Western tropical zodiac boundaries only.** Sidereal/Vedic zodiac systems and alternative astrological traditions are out of scope.
- **Static trait vocabulary.** Strength/challenge phrasing is drawn from a fixed `trait_strength/2` / `trait_challenge/2` table; it does not adapt to combinations of traits beyond simple aggregation.
- **LLM dependency for narrative depth.** Rich, varied prose depends on OpenRouter availability; the deterministic fallback is functionally correct but noticeably more templated and repetitive.
- **Single-language support.** The knowledge base and generated content are English-only; no localization layer exists yet.
- **No user accounts / multi-tenant isolation.** Reading history is stored in a local SQLite database without authenticated, per-user separation.
- **Compatibility model is symbolic, not empirical.** The weighted scoring formula encodes traditional astrological symbolism; it is not validated against psychological or relationship-outcome data, and is presented purely as a reflective tool.
- **Question classification is heuristic.** Category detection uses keyword/pattern-based NLP rather than a trained classifier, so ambiguous or multi-topic questions can be mis-categorized.

7. System Implementation (UI Design)

7.1 Application Pages

Route	Purpose
`/dashboard`	Landing overview – quick access to readings, horoscope, and recent history.
`/zodiac`	Browse all 12 signs; view element, modality, ruling planet, traits, strengths and challenges.
`/birth-chart`	Enter a birth date to resolve a Sun-sign profile.
`/tarot`	Browse the 78-card deck; view keywords and upright/reversed meanings.
`/reading`	Ask a question, receive a recommended spread, draw cards, and view the AI-narrated interpretation.
`/horoscope`	Generate a themed daily horoscope from sign and mood.
`/compatibility`	Compare two zodiac signs and view the weighted score breakdown and explanation.
`/chat`	Conversational AI assistant grounded in Prolog facts and reading context.
`/reasoning`	Dedicated reasoning-trace explorer – visualizes the rule chain behind a result.
`/history`	Review, revisit, and delete past readings and chat sessions.
`/analytics`	Aggregate statistics across a user's reading history.
`/scan`	Auxiliary input/scan flow feeding into a reading.
`/about`	Project description and ethical framing.

7.2 Key UI Components

- `TarotCard` – renders card art, name, orientation, and meaning.
- `ZodiacBadge` / `ElementBadge` – compact visual tags for sign and element.
- `ReasoningStep` – renders one entry of a Prolog reasoning trace (rule, input, result, explanation).

- `LoadingSpinner`, `StarField`, `Navigation` – shared visual/interaction chrome.

State is managed through dedicated Svelte stores – `Profile`, `Reading`, `Chat`, and `Loading` – with a typed API client that centralizes error handling for all backend calls.

7.3 REST API Surface

```
GET  /api/health
GET  /api/zodiac
GET  /api/zodiac/{sign}
GET  /api/tarot
POST /api/tarot/draw          { count, spread_type }
POST /api/reading/analyze     { question, zodiac_sign, spread_type }
POST /api/reading/generate
POST /api/horoscope/generate  { zodiac_sign, mood }
POST /api/compatibility/analyze  { sign1, sign2 }
POST /api/chat                { message, zodiac_sign, current_reading }
GET    /api/history
GET    /api/history/{id}
DELETE /api/history/{id}
```

Base URL: `http://localhost:8000/api` *(from* `docs/api.md`*).*

7.4 Backend Service Layer

FastAPI is organized into `api` (route handlers), `services` (`PrologService`, `OpenRouterService`, `TarotService`, `HoroscopeService`, `AnalyticsService`), `schemas` (Pydantic request/response models), `models`, and `database` (SQLite access). `PrologService` is the single bridge between Python and SWI-Prolog, exposing typed methods for every predicate in each of the four reasoning modules (e.g. `get_zodiac_profile`, `select_eligible_cards`, `analyze_reading`, `analyze_synastry`).

7.5 Visual/UX Design Notes

- Dark, star-field themed interface (via the `StarField` component) consistent with the astrology/tarot subject matter.
- Reasoning is never hidden: every generated result links to a reasoning-trace view so users can see *why* the system produced that answer.
- Responsive layout built with Tailwind CSS utility classes; component styles kept local to each Svelte component.

8. Conclusion

AstroLogic demonstrates that a large, traditionally free-text domain – astrology and tarot – can be modeled as a structured, rule-based reasoning system. By encoding zodiac and tarot knowledge as declarative Prolog facts and layering four purpose-built reasoning modules on top (profile derivation, card selection, reading analysis, and synastry scoring), the project produces results that are **explainable and reproducible**: every output can be traced back to the specific facts and rules that generated it. The LLM is deliberately confined to natural-language presentation, which keeps the system's actual "intelligence" auditable rather than opaque, and provides a robust deterministic fallback path. Combined with a modern SvelteKit interface and a documented FastAPI backend, AstroLogic delivers a coherent, end-to-end example of symbolic AI applied to an engaging, user-facing product – and shows in practice how classical logic-programming techniques (fact bases, Horn-clause rules, weighted scoring, cut-guarded lookup) remain a powerful, transparent complement to modern generative AI.

9. Further Extension

- **Full natal chart support.** Extend `chart_profile/4` with ephemeris data to compute Moon sign, Rising sign, and planetary house placements from birth time and location.
- **Trained question classifier.** Replace the heuristic category detector with a lightweight trained NLP model for more accurate spread recommendation.
- **Personalization over history.** Use `history_analysis.pl` more extensively to surface long-term patterns (recurring themes, sign trends) back to the user through the Analytics page.
- **Multi-language support.** Localize both the Prolog trait/theme text and the LLM prompts for non-English users.
- **User accounts and multi-tenancy.** Add authentication so reading history is securely scoped per user.
- **Alternative zodiac systems.** Add optional sidereal/Vedic rule sets alongside the existing Western tropical rules.
- **Expanded compatibility model.** Incorporate additional symbolic factors (e.g. house overlays, aspect angles) once full chart data is available.
- **Offline/edge deployment.** Package the Prolog engine and a small local LLM together for an offline-capable version of the app.

10. References

1. SWI-Prolog Reference Manual. *SWI-Prolog Development Team.* https://www.swi-prolog.org/pldoc/doc_for?object=manual
2. Bratko, I. *PROLOG Programming for Artificial Intelligence*, 4th ed. Pearson, 2011.
3. Russell, S., & Norvig, P. *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2020.
4. FastAPI Documentation. https://fastapi.tiangolo.com/
5. SvelteKit Documentation. https://svelte.dev/docs/kit
6. OpenRouter API Documentation. https://openrouter.ai/docs
7. AstroLogic Project Documentation – `docs/architecture.md`, `docs/prolog-reasoning.md`, `docs/ai-integration.md`, `docs/api.md`, `docs/database.md` (internal project repository).